```c
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1) return 1;

    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n], completed[n];
    float total_wt = 0, total_tat = 0;

    // Input Loop
    for(int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("\n--- Process P%d ---\n", pid[i]);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority (Lower = Higher Priority): ");
        scanf("%d", &pr[i]);
        completed[i] = 0;
    }

    int current_time = 0;
    int completed_count = 0;

    while(completed_count < n) {
        int highest_priority = 9999;
        int selected_process = -1;

        for(int i = 0; i < n; i++) {
            // Check if process has arrived and isn't finished
            if(at[i] <= current_time && completed[i] == 0) {
                if(pr[i] < highest_priority) {
                    highest_priority = pr[i];
                    selected_process = i;
                }
            }
        }

        if(selected_process == -1) {
            // CPU is idle
            current_time++;
```

```
        } else {
            // Calculate Timings
            ct[selected_process] = current_time + bt[selected_process];
            tat[selected_process] = ct[selected_process] - at[selected_process];
            wt[selected_process] = tat[selected_process] - bt[selected_process];

            total_wt += wt[selected_process];
            total_tat += tat[selected_process];

            current_time = ct[selected_process];
            completed[selected_process] = 1;
            completed_count++;
        }
    }

    // Table Output
    printf("\n%-10s %-5s %-5s %-5s %-5s %-5s %-5s\n", "Process", "AT", "BT", "PR", "CT",
"TAT", "WT");
    for(int i = 0; i < n; i++) {
        printf("P%-9d %-5d %-5d %-5d %-5d %-5d %-5d\n",
               pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Waiting Time: %.2f", total_wt / n);
    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

    return 0;
}
```

```
Process P2
Arrival Time: 2
Burst Time: 2
Priority: 3

Process P3
Arrival Time: 3
Burst Time: 5
Priority: 2

Process P4
Arrival Time: 4
Burst Time: 4
Priority: 4

Process P5
Arrival Time: 6
Burst Time: 1
Priority: 1

Process AT      BT      PR      CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       11      9       7
P3      3       5       2       8       5       0
P4      4       4       4       15      11      7
P5      6       1       1       9       3       2

Process returned 0 (0x0)   execution time : 47.018 s
Press any key to continue.
```

```c
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n], rt[n];
    int completed = 0, current_time = 0;

    // Input Process Details
    for(int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("\nProcess P%d\n", pid[i]);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &pr[i]);
        rt[i] = bt[i]; // Initialize remaining time
    }

    // Scheduling Logic
    while(completed < n) {
        int highest_priority = 9999;
        int selected = -1;

        for(int i = 0; i < n; i++) {
            // Check if process has arrived and isn't finished
            if(at[i] <= current_time && rt[i] > 0) {
                if(pr[i] < highest_priority) {
                    highest_priority = pr[i];
                    selected = i;
                }
                // Tie-breaker: If priorities are equal, pick the one that arrived first (FCFS)
                else if(pr[i] == highest_priority) {
                    if(at[i] < at[selected]) {
                        selected = i;
                    }
                }
            }
        }
```

```
        if(selected == -1) {
            current_time++; // CPU is idle
        } else {
            rt[selected]--; // Execute for 1 unit of time
            current_time++;

            if(rt[selected] == 0) {
                completed++;
                ct[selected] = current_time;
                tat[selected] = ct[selected] - at[selected];
                wt[selected] = tat[selected] - bt[selected];
            }
        }
    }

    // Display Results
    printf("\n%-7s %-5s %-5s %-5s %-5s %-5s %-5s\n", "Process", "AT", "BT", "PR", "CT", "TAT",
"WT");
    for(int i = 0; i < n; i++) {
        printf("P%-6d %-5d %-5d %-5d %-5d %-5d %-5d\n",
               pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

```
Burst Time: 1
Priority: 5

Process P5
Arrival Time: 5
Burst Time: 6
Priority: 2

Process P6
Arrival Time: 6
Burst Time: 5
Priority: 6

Process P7
Arrival Time: 10
Burst Time: 1
Priority: 1

Process AT      BT      PR      CT      TAT     WT
P1      0       8       3       15      15      7
P2      1       2       4       17      16      14
P3      3       4       4       21      18      14
P4      4       1       5       22      18      17
P5      5       6       2       12      7       1
P6      6       5       6       27      21      16
P7      10      1       1       11      1       0

Process returned 0 (0x0)   execution time : 82.669 s
Press any key to continue.
```

```c
#include <stdio.h>

int main() {
    int n, tq, time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], rt[n], wt[n], tat[n], finish[n];
    for(int i = 0; i < n; i++) finish[i] = 0; // Track completed processes

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    printf("\nEnter Process ID, Arrival Time, Burst Time:\n");
    for(int i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d %d %d", &pid[i], &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    // Scheduling Logic
    while (completed < n) {
        int executed_in_cycle = 0;

        for (int i = 0; i < n; i++) {
            if (rt[i] > 0 && at[i] <= time) {
                executed_in_cycle = 1;
                if (rt[i] > tq) {
                    time += tq;
                    rt[i] -= tq;
                } else {
                    time += rt[i];
                    wt[i] = time - at[i] - bt[i];
                    tat[i] = time - at[i];
                    rt[i] = 0;
                    completed++;
                }
            }
        }

        // If no process was ready at the current time, jump to the next arrival
        if (executed_in_cycle == 0) {
```

```
            time++;
        }
    }

    // Output Results
    printf("\nPID\tAT\tBT\tWT\tTAT\n");
    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], wt[i], tat[i]);
        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

```
Enter number of processes: 5
Enter Time Quantum: 2

Enter Process ID, Arrival Time, Burst Time (in order)
Process 1: 1
0
5
Process 2: 2
1
3
Process 3: 3
2
1
Process 4: 4
3
2
Process 5: 5
4
3

PID      AT       BT       WT       TAT
1        0        5        9        14
2        1        3        8        11
3        2        1        2        3
4        3        2        2        4
5        4        3        6        9

Average Waiting Time = 5.40
Average Turnaround Time = 8.20
```